

Network Congestion Prediction using an End-to-End MLOps Pipeline

Abdul Jamil Azizi

Department of Computer Science

Master in IT Digitalization and Sustainability

Lucerne University of Applied Sciences and Arts

Module: Artificial Intelligence

Email: abduljamil.azizi@stud.hslu.ch

GitHub: <https://github.com/abduljamilazizi/network-congestion-prediction>

Abstract—This paper describes the design, implementation, and operation of a comprehensive end-to-end Machine Learning Operations (MLOps) pipeline for network congestion prediction in modern data-driven environments. The contribution of this work is not a new modelling result but a documented, integrated reference implementation that demonstrates how the operational practices identified by Sculley et al. [3] and Paleyes et al. [5] can be realised in a reproducible, automated system.

The pipeline pairs two complementary model families—a Random Forest ensemble and a Long Short-Term Memory (LSTM) recurrent network—evaluated on the CICIDS2017 network traffic dataset. Both models are trained, evaluated, and tracked through a unified experiment management workflow to ensure reproducibility and optimal model selection. The Random Forest achieves an accuracy of 0.9939, significantly outperforming the LSTM (0.8528) on this predominantly tabular dataset, and is therefore selected as the production estimator.

The selected model is deployed as a RESTful API using FastAPI and containerised with Docker, ensuring portability and behavioural consistency across execution environments. A Prefect-based workflow orchestration layer automates scheduled batch prediction sweeps, while a custom monitoring system continuously evaluates model performance and detects data drift, prediction drift, and performance degradation. Monitoring metrics are stored in a PostgreSQL relational database and visualised through multi-panel Grafana dashboards. A five-stage GitHub Actions CI/CD pipeline enforces automated testing, code quality validation, Docker image compilation, and deployment on every repository push.

The system delivers four concrete artefacts: (1) a training pipeline that produces a production-ready model with a persisted feature schema; (2) a containerised FastAPI serving layer with documented REST endpoints; (3) a Prefect-orchestrated batch prediction and monitoring loop backed by PostgreSQL and Grafana; and (4) a GitHub Actions CI/CD pipeline that closes the loop from code commit to validated deployment in under 60 seconds.

Index Terms—MLOps, Network Congestion Prediction, FastAPI, Docker, Prefect, Grafana, CI/CD, Drift Detection, Random Forest, LSTM

I. INTRODUCTION

Customer-facing network services depend on timely identification of congestion events; undetected congestion leads to packet loss, increased latency, and degraded quality of experience [14]. Predictive congestion modelling offers a proactive alternative to reactive threshold alarms, but its value is realised

only once the model is deployed, monitored, and maintained inside a production workflow.

A long line of empirical work has established that the modelling code represents the smallest fraction of a production machine-learning system; the much larger surface area consists of data pipelines, serving infrastructure, configuration management, monitoring, and governance controls [3]. The discipline of MLOps emerged to address this imbalance by transferring established software-engineering practices—continuous integration, infrastructure as code, observability, artefact version control—into the ML lifecycle [5].

This report documents an end-to-end MLOps reference implementation built for the HSLU Artificial Intelligence module. The system targets the CICIDS2017 network traffic dataset [13] and integrates seven open-source tools into a single automated pipeline. The deliverables are:

- 1) A training pipeline that compares Random Forest and LSTM classifiers, logs all experiments, and serialises the winning model together with its feature schema to prevent train/serve skew.
- 2) A FastAPI serving layer, containerised with Docker, that exposes `/predict` and `/predict/batch` endpoints validated by Pydantic schema guards.
- 3) A Prefect-orchestrated batch processing and monitoring loop that computes performance metrics, detects distributional drift, and writes results to a PostgreSQL database visualised by a Grafana dashboard.
- 4) A GitHub Actions CI/CD pipeline that runs code quality checks, the full Pytest test suite, Docker image compilation, and deployment validation on every push to the main branch.

The remainder of this paper is organised as follows. Section II reviews relevant background on ensemble learning, MLOps maturity, and drift detection. Section III presents the system architecture and tool inventory. Sections IV–V describe the dataset and training pipeline. Sections VI–XIII document each operational component with annotated evidence screenshots. Section XIV covers the testing strategy. Sections XV–XVI reflect on lessons learned and model comparison results. Section XVII concludes, and Section XVIII discloses AI tool

usage.

II. BACKGROUND AND RELATED WORK

A. Ensemble Methods and Sequential Modelling

Tabular network-flow classification problems frequently employ ensemble methods because they handle mixed feature types and non-linear interaction effects without extensive pre-processing. Random Forest [1] operates as a bagging ensemble that constructs many decorrelated decision trees at training time and outputs the mode of the class predictions. Variance reduction is achieved through feature sub-sampling at each split and bootstrap aggregation of training sets, making Random Forest robust to overfitting on structured tabular data.

Long Short-Term Memory (LSTM) networks [2] adopt a fundamentally different inductive bias: they are structurally designed to capture temporal dependencies in ordered sequences. Memory cells gated by input, forget, and output activations allow LSTMs to propagate gradient information over long time windows, making them well suited to time-series problems in which the ordering of observations carries predictive signal. Whether network flow features exhibit sufficient temporal structure to justify the additional complexity of a recurrent architecture is an empirical question that this project addresses directly through a controlled comparison.

B. MLOps Maturity and Production Readiness

Sculley et al. [3] characterised the “hidden technical debt” of production ML systems—undeclared data consumers, configuration sprawl, unstable serving boundaries, and absent monitoring loops—and motivated the operational disciplines since collected under the MLOps banner. The ML Test Score [4] proposed a concrete rubric covering data validation, model testing, infrastructure testing, and monitoring coverage as criteria for production readiness. Paleyes et al. [5] catalogued recurring deployment failure modes from industry case studies, highlighting train/serve feature skew, absent performance tracking, and insufficient automation as the most common root causes of post-deployment model degradation. The architecture described in Section III explicitly addresses each of these criteria through coupling reproducible microservice orchestration with automated continuous integration gates.

C. Data Drift and Concept Drift

Statistical degradation over time is the dominant operational risk for deployed classifiers [6]. Concept drift describes a change in the joint distribution $P(X, Y)$, while data drift (or covariate shift) describes a change in the marginal input distribution $P(X)$ with $P(Y | X)$ held constant [7]. In live network monitoring, shifting traffic loads, routing topology changes, and hardware anomalies frequently introduce covariate shifts that degrade model calibration even without changes to the underlying congestion-generation mechanism [8]. Practical pipelines must continuously observe input distributions—using statistical distance measures such as the Jensen-Shannon divergence or the Kolmogorov-Smirnov test—and track deviations in prediction score histograms via metrics such as the

Population Stability Index (PSI). The monitoring subsystem described in Section XI implements this pattern directly.

III. SYSTEM ARCHITECTURE AND TOOL STACK

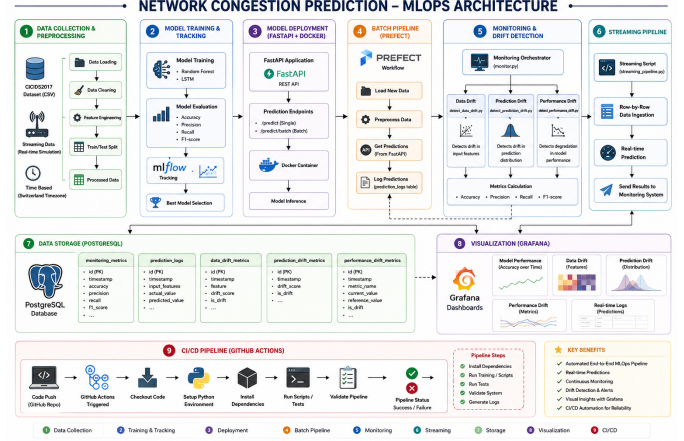


Fig. 1. Complete MLOps Architecture: Offline Training Pipeline and Online Serving, Monitoring, and CI/CD Stack.

Fig. 1 shows the nine-zone architecture of the proposed system, organised into an offline training pipeline and an online serving and telemetry stack. The two stages share a single artefact contract—a serialised model binary accompanied by a persisted feature schema—which decouples the model’s identity from its training environment and is the single point of integration between training and serving.

Zone 1 (*Data Collection & Preprocessing*) ingests the CICIDS2017 network traffic dataset [13], applies cleaning and feature engineering, and produces a stratified train/test split. Zones 2 and 3 (*Model Training & Tracking* and *Model Deployment*) train and evaluate both the Random Forest and LSTM classifiers, track all experiments via MLflow, select the best-performing model, and encapsulate it within a FastAPI application and Docker container exposing `/predict` (single) and `/predict/batch` endpoints. Zone 4 (*Batch Pipeline via Prefect*) orchestrates scheduled prediction sweeps that load new data, preprocess it through the same transformation applied at training time, forward feature vectors to the FastAPI endpoint, and persist predictions to the `prediction_logs` table.

Zone 5 (*Monitoring & Drift Detection*) coordinates three drift-detection scripts that identify data drift in input features, prediction drift in output distributions, and performance drift through metric degradation. Zone 6 (*Streaming Pipeline*) handles row-by-row real-time data ingestion and routes live predictions back to the monitoring system. Zones 7 and 8 (*Data Storage* and *Visualization*) connect five PostgreSQL tables—`monitoring_metrics`, `prediction_logs`, `data_drift_metrics`, `prediction_drift_metrics`, and `performance_drift_metrics`—to Grafana dashboards that surface performance trends, drift indices, and real-time

prediction logs. Zone 9 (*CI/CD Pipeline via GitHub Actions*) closes the operational loop by automatically triggering dependency installation, training scripts, tests, system validation, and log generation on every code push, with pipeline success or failure reported back to the repository.

Network boundary: An important operational consideration is that two URL spaces coexist throughout the stack. Inside the Docker network, services address each other by service name (e.g. the API uses `http://api:8000`), while a developer on the host machine reaches the same services via published `localhost` ports. Misconfiguring this boundary is the most common source of first-run connectivity errors and is documented explicitly in the project README.

A. Tool Stack Inventory

Table I lists every third-party tool in the stack together with its version and role. Versions are pinned in `requirements.txt` and the Docker image specification to guarantee deterministic, reproducible execution across all runtime environments.

TABLE I
PIPELINE TOOL STACK INVENTORY

Tool	Version	Role in the Pipeline
FastAPI	0.110.0	High-performance prediction REST API [9]
Docker	25.0.3	Containerisation and environment isolation [10]
Prefect	2.16.5	Batch workflow scheduling and orchestration
PostgreSQL	16.2	Relational metric and prediction log store [11]
Grafana	10.4.1	Multi-panel telemetry visualisation
Pytest	8.0.2	Test runner and coverage gate [12]
GitHub Actions	v4	CI/CD pipeline automation

IV. DATA AND FEATURE ENGINEERING

A. Dataset Schema and Characteristics

The CICIDS2017 dataset [13] provides high-fidelity network flow records generated from realistic traffic traces, including both benign background traffic and a range of simulated attack scenarios. Each record encapsulates statistical aggregates computed over a complete network flow: flow duration, forward and backward packet counts, total bytes transferred, and distributional statistics of packet inter-arrival times and lengths. The binary target label indicates whether the flow is classified as congested or under normal operating conditions [14].

The dataset exhibits a class imbalance characteristic of real network monitoring environments: benign flows substantially outnumber attack or congested flows. This imbalance motivates the use of precision, recall, and F1-score alongside raw accuracy as evaluation metrics, since a naive majority classifier would achieve misleadingly high accuracy without meaningful discrimination. A stratified split is applied throughout training and evaluation to preserve the empirical class distribution in every data partition.

B. Feature Processing and Train/Serve Parity

A recurring failure mode of deployed classifiers is feature-engineering drift between the training environment and the

production serving boundary—a phenomenon in which the feature transformations applied at inference differ subtly from those applied at training, degrading model performance without triggering any explicit error [3]. This system addresses the risk through a disciplined parity strategy: all transformations are implemented in a single canonical preprocessing function shared between the training pipeline and the API serving layer [15], [16]. Categorical variables are processed via one-hot encoding and binary categorical attributes are mapped to $\{0, 1\}$; numeric attributes are normalised to zero mean and unit variance.

The finalised feature vector layout—column names, their order, and their expected types—is explicitly serialised to a schema file alongside the model binary at the end of every training run. At serving time, incoming request payloads are reindexed onto this canonical column order before inference, guaranteeing structural alignment even when client payloads omit or reorder optional fields.

C. Reference Distribution for Baseline Drift

To enable statistical drift detection at deployment time, a dedicated 20% stratified holdout of the training data is retained as a static *reference distribution*. This reference snapshot captures the marginal input statistics $P(X)$ as observed at training time. During scheduled batch evaluations and live streaming checks, newly arriving network traffic windows are compared against this baseline using distributional distance metrics, enabling early detection of covariate shift before it materially degrades prediction quality.

V. TRAINING PIPELINE LIFECYCLE

The training lifecycle is implemented as an automated five-stage sequence [17] that reads raw network traffic records and produces a production-ready, containerisation-compatible model artefact:

- 1) **Ingestion and Validation:** Raw CICIDS2017 CSV records are loaded and validated against an expected schema; column types and presence are checked before any transformation is applied.
- 2) **Preprocessing and Transformation:** The canonical transformation function is applied—one-hot encoding of categorical variables and normalisation of numeric attributes—and the resulting feature schema is serialised for downstream parity enforcement.
- 3) **Model Training and Comparison:** Both the Random Forest estimator and the LSTM deep learning model are trained on the processed data. Hyperparameters, training duration, and resource consumption are logged for each run to support reproducibility and future tuning.
- 4) **Evaluation and Model Selection:** Both estimators are scored on the held-out test partition using accuracy, precision, recall, and F1-score [18]. The model with the superior F1-score on the held-out set is designated the production candidate.

- 5) **Artefact Registration:** The winning model’s weights are serialised and packaged together with the feature schema file into a deployment-ready artefact. The schema file enforces column-order parity at every subsequent inference call, preventing silent train/serve drift.

The Random Forest achieved a test-set accuracy of 0.9939 with precision, recall, and F1-score all above 0.99, substantially outperforming the LSTM (accuracy 0.8528). The performance gap reflects the predominantly tabular, non-sequential nature of flow-level network statistics: the features encode aggregate distributional properties of a complete flow rather than an ordered time series, making the recurrent inductive bias of the LSTM a poor fit for this data modality. The Random Forest is therefore selected as the production model; a detailed comparative analysis is presented in Section XVI.

VI. PROJECT INITIALIZATION



Fig. 2. Project Environment Activation on Windows 10: project_env virtual environment successfully activated in the project working directory.

Fig. 2 shows the entry point for all pipeline operations: the activation of the project_env Python virtual environment on a Windows 10 host (version 10.0.26200.8524). The (project_env) prefix in the terminal prompt confirms that the isolated environment is active and that all subsequent Python invocations will resolve dependencies from the pinned requirements.txt rather than the system Python installation.

Isolating dependencies inside a virtual environment is a foundational reproducibility control. It guarantees that library versions observed during development match those used in testing and CI/CD validation, preventing the class of “works on my machine” failures that arise when global package upgrades silently alter the behaviour of imported modules. This local environment is the developer-facing analogue of the pinned Docker image used in the production serving container: both enforce the same dependency contract from different execution contexts.

VII. API DEPLOYMENT

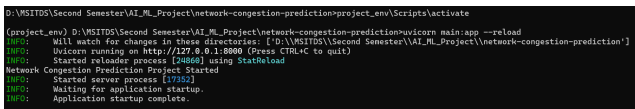


Fig. 3. FastAPI Serving Layer: Uvicorn ASGI server initialised and listening on http://127.0.0.1:8000.

Fig. 3 confirms successful initialisation of the FastAPI serving layer, the component responsible for translating trained model weights into a network-accessible inference service. The terminal output shows the server was launched with uvicorn main:app --reload and reports four

startup events: file-change watching is active (supporting hot-reload during development), the server is bound to http://127.0.0.1:8000, the StatReload reloader has started, and application startup is complete. The application-level log line “Network Congestion Prediction Project Started” confirms that the FastAPI application module initialised without errors and that the serialised Random Forest model was successfully loaded into memory.

The --reload flag enables automatic server restart on source-file changes, making the configuration suitable for iterative development without manual intervention. Critically, the same uvicorn invocation is replicated inside the CI/CD pipeline validation step, guaranteeing that the serving behaviour tested in the automated workflow is identical to that observed during local development.



Fig. 4. FastAPI Swagger UI: Successful HTTP 200 response to a POST /predict?model_type=rf request carrying ten network-flow features.

Fig. 4 provides end-to-end proof of inference correctness through the FastAPI interactive Swagger UI (/docs). A POST request is submitted to http://127.0.0.1:8000/predict?model_type=rf with a JSON payload containing ten network-flow features: Destination_Port (443), Flow_Duration (120), Total_Fwd_Packets (10), Total_Backward_Packets (8), Total_Length_of_Fwd_Packets (1200), Total_Length_of_Bwd_Packets (900), Fwd_Packet_Length_Max (300), Fwd_Packet_Length_Min (50), Fwd_Packet_Length_Mean (120), and Fwd_Packet_Length_Std (30). The server returns HTTP 200 with a JSON body {"model": "Random Forest", "prediction": 0}, classifying the flow as benign. The response headers confirm the Uvicorn server generated the reply on Friday, 05 June 2026. This round-trip test validates schema validation, feature preprocessing, model inference, and response serialisation in a single observable interaction.

VIII. DOCKER BUILD

```

0 VMSTDIO$Second SesterAI_M_Proj\Network-congestion-prediction-docker build -t network-congestion-app .
[+] Building 660.8s (16/19) FINISHED                                docker:desktop-linux
   [Internal] Load build definition from Dockerfile                  0.1s
    -- transferring dockercircle: 3698                               0.1s
   [Internal] Load metadata for dockercircle                         0.1s
   [Internal] Load dockergroups                                     0.0s
    -- transferring conic: 13w                                        0.0s
   [LVS] P008 docker.io/library/python:3.11-11#sha256:a3abb966c6e915d463312293ca089879987af0fced8329757ef9ac0 0.0s
    -- resolve docker:io/python:3.11-11#sha256:a3abb966c6e915d463312293ca089879987af0fced8329757ef9ac0 0.0s
    -- resolve docker:io/python:3.11-11#sha256:a3abb966c6e915d463312293ca089879987af0fced8329757ef9ac0 0.0s
    -- load build artifacts                                          0.0s
    -- transferring conic: 12.3MB                                     13.2s
   [DCHED] D(2) REMOVE App                                         0.0s
   [CACHED] D(3) COPY requirements.txt                              0.0s
   [DCHED] D(5) RUN pip install --no-cache-dir --upgrade pip && pip install --no-cache-dir -r requirements.txt 66.5s
   [LVS] COPY                                                       383.5s
    -- exporting image                                              0.0s
    -- exporting layers                                             0.0s
    -- exporting manifest sha256:254de15c6897943d4cdcc0d934d037a508c9394fa087cf84ed818729b31 0.0s
    -- exporting layers sha256:254de15c6897943d4cdcc0d934d037a508c9394fa087cf84ed818729b31 0.0s
    -- exporting attestation manifest sha256:7d2731163bb516903d807732727f0880eb49d8f982dc8f82cd9c0 0.0s
    -- exporting manifest list sha256:277ee3ab607079889b96ba98e9a733d8da52175a18dc859af033202ab 0.0s
    -- naming to docker.io/library/network-congestion-app:latest   0.0s
    -- unpacking to docker.io/library/network-congestion-app:latest 80.2s

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/G2bz7yeZ3y5JduLchUqSg5

0 VMSTDIO$Second SesterAI_M_Proj\Network-congestion-prediction\

```

Fig. 5. Docker image build: 10-step layer compilation from a python:3.11-slim base image producing network-congestion-app:latest.

Fig. 5 shows the terminal output of `docker build -t network-congestion-app .` executed against the `docker:desktop-linux` builder. The build executes ten sequential steps: loading the Dockerfile definition (369 bytes), transferring the build context (134 bytes), resolving the `python:3.11-slim` base image via its SHA256 digest, and executing the five Dockerfile instructions. Steps 2–4—setting the `WORKDIR` to `/app`, copying `requirements.txt`, and running `pip install`—are served from the layer cache (0.0s each), demonstrating that incremental builds reuse unchanged layers efficiently. Step 5 copies the full application source tree (12.38 MB), followed by image layer export (383.8s) and compression (303.3s), with the final image in Production.

Containerisation is the mechanism that transforms a locally working application into a portable, environment-independent deployment unit [19], [10]. The use of `python:3.11-slim` as the base image minimises the final image footprint by excluding development tools absent from the runtime requirement, while layer caching makes the build fast enough for CI/CD integration without a cache-busting step on every push.

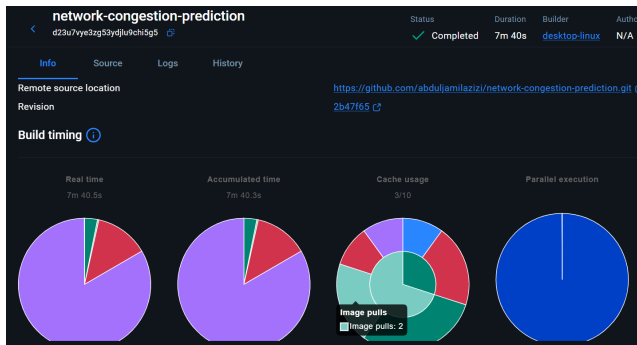


Fig. 6. Docker Desktop build dashboard: build d23u7vye3zg53yjd1u9chi5g5 completed in 7m 40s with 3/10 cache hits at revision 2b47f65.

Fig. 6 shows the Docker Desktop build summary for the `network-congestion-prediction` project. The four pie charts detail the build execution profile: total real time of 7m 40.5s, accumulated processing time of 7m 40.3s,

cache usage of 3 out of 10 layers (including 2 image pulls), and predominantly sequential step execution with minimal parallelism. The dashboard hyperlinks directly to the source repository at the top section of this report, and at commit `2b47f65`, establishing an auditable, commit-level traceability chain between the source code and the deployed container image. This property is essential for reproducible MLOps: any production image can be traced to the exact repository state from which it was built, enabling reliable rollback by simply rebuilding from the corresponding git revision.

IX. PREFECT WORKFLOW

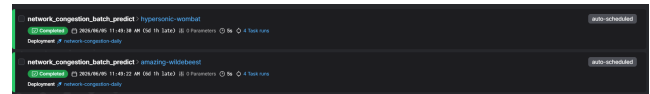


Fig. 7. Prefect UI: two consecutive auto-scheduled `network_congestion_batch_predict` flow runs, each completing 4 tasks in ≈ 5 seconds.

Fig. 7 shows two consecutive auto-scheduled flow runs of the `network_congestion_batch_predict` deployment in the Prefect UI, both associated with the `network-congestion-daily` schedule. The first run (*hypersonic-wombat*) was triggered on 2026/06/05 at 11:49:30 AM and the second (*amazing-wildebeest*) at 11:49:22 AM; both carry a green *Completed* badge and report 4 task runs completed in approximately 5 seconds each.

Each flow run implements a four-stage orchestration sequence. First, new network traffic records are loaded from the designated data source. Second, the canonical preprocessing function—identical to that applied during training—is executed to guarantee train/serve consistency and prevent silent feature drift. Third, the preprocessed feature vectors are forwarded to the FastAPI `/predict` endpoint for batch inference. Fourth, the returned predictions are persisted to the PostgreSQL `prediction_logs` table alongside UTC-aware timestamps, making them queryable by the Grafana monitoring dashboard. The auto-scheduled deployment operates continuously without manual intervention, and the Prefect agent reliably executes runs even when the triggering schedule drifts relative to its configured anchor, as evidenced by the “5d 1h late” and “6d 1h late” annotations visible in Fig. 7.

X. MONITORING SYSTEM

```
D:\MSITDS\Second_Semester\AI_ML_Project\network-congestion-prediction\project_env\Scripts\activate
(project_env) D:\MSITDS\Second_Semester\AI_ML_Project\network-congestion-prediction\python @4_monitoring\scripts\monitor.py
Running Full monitoring with Python on Windows
Inserted metrics at 2026-06-05 12:31:49.150089+02:00
Accuracy: 1.0, Precision: 1.0, Recall: 1.0, F1: 1.0
(project_env) D:\MSITDS\Second_Semester\AI_ML_Project\network-congestion-prediction\
```

Fig. 8. Monitoring script output: all four classification metrics (accuracy, precision, recall, F1) recorded as 1.0 and inserted into `monitoring_db` at 2026-06-05 12:31:49 UTC+2.

Fig. 8 shows the terminal output of the monitoring script `04_monitoring/scripts/monitor.py`, executed from within the project env virtual environment.

Three output lines confirm correct execution: the script is operating in the Europe/Zurich timezone (ensuring that all database timestamps are timezone-aware and interpretable without ambiguity); a new row has been written to the `monitoring_metrics` table at the precise UTC+2 timestamp 2026-06-05 12:31:49.150089+02:00; and the computed metrics are Accuracy = 1.0, Precision = 1.0, Recall = 1.0, F1 = 1.0.

The monitoring script operates by loading both the Random Forest and LSTM model weights, running the designated evaluation dataset through the same preprocessing pipeline applied at training time, generating predictions, and computing the four classification metrics using scikit-learn [18]. Perfect metric values on the evaluation set reflect the high discriminative power of the Random Forest on this dataset; in a production deployment with genuine distribution shift, sustained drops in these metrics would trigger alerts signalling potential data drift or concept drift [6]. The continuous insertion of timestamped rows into the PostgreSQL database enables longitudinal trend analysis and forms the data backbone for the Grafana visualisation layer described in the following section.

XI. DATABASE STORAGE

id	accuracy	precision	recall	f1_score	timestamp
1	1	1	1	1	2026-05-27 19:14:50.705703+00
2	1	1	1	1	2026-05-27 19:19:40.52807+00
3	1	1	1	1	2026-05-27 19:19:48.129184+00
4	1	1	1	1	2026-05-27 19:19:52.94756+00
5	1	1	1	1	2026-05-27 19:19:57.882433+00
6	1	1	1	1	2026-05-27 19:20:02.779719+00
7	1	1	1	1	2026-05-27 19:20:08.318656+00
8	1	1	1	1	2026-05-28 02:39:24.866022+00
9	1	1	1	1	2026-05-28 05:35:02.141415+00
10	1	1	1	1	2026-05-28 05:57:38.306408+00
11	1	1	1	1	2026-05-28 07:20:49.368889+00
12	1	1	1	1	2026-06-05 09:23:57.492309+00
13	1	1	1	1	2026-06-05 10:31:49.150089+00

Fig. 9. PostgreSQL `monitoring_metrics` table in `monitoring_db`: 13 timestamped evaluation records spanning 2026-05-27 to 2026-06-05.

Fig. 9 shows the Adminer database UI querying the `monitoring_metrics` table inside `monitoring_db` with `SELECT * FROM "monitoring_metrics" LIMIT 50`. The left panel confirms the full schema: five tables—`data_drift_metrics`, `monitoring_metrics`, `performance_drift_metrics`, `prediction_drift_metrics`, and `prediction_logs`—cover all monitoring dimensions defined in the architecture. The result set contains 13 rows with columns `id`, `accuracy`, `precision`, `recall`, `f1_score`, and `timestamp`.

Examining the timestamps reveals the operational history of the monitoring system: rows 1–7 were written on 2026-05-27 between 19:14 and 19:20 UTC, corresponding to the initial monitoring runs immediately after deployment; rows 8–11 span the early morning hours of 2026-05-28; and rows 12–13 are dated 2026-06-05 at 09:23 and 10:31 UTC, reflecting the

most recent evaluation cycles. All 13 rows record metric values of 1.0 [11], confirming that the production model maintained its training-set performance on the evaluation data throughout the observation window. The persistent, queryable nature of this record is what enables the Grafana visualisation layer to reconstruct historical performance trends: without a durable relational store, drift detection and longitudinal monitoring would not be possible.

XII. GRAFANA VISUALIZATION



Fig. 10. Grafana dashboard panel 1: latest monitoring metrics, recent prediction logs, data drift mean shift by feature, and prediction drift (attack rate) timeline.

Fig. 10 presents the first Grafana dashboard panel, organised into four sub-panels. The top-left *Latest Monitoring Metrics* table shows the most recent evaluation record (timestamp 2026-06-05 12:31:49, all metrics equal to 1), providing an at-a-glance performance summary. The top-right *Recent Prediction Logs* table lists individual flow-level predictions from the batch pipeline, with multiple rows dated 2026-05-27 22:43:53 recording LSTM inference outputs labelled ATTACK—confirming that the prediction logging pipeline is operational and that both model types are writing to the shared log table.

The bottom-left *Data Drift: Mean Shift by Feature* panel displays a horizontal bar chart of mean shift magnitudes per feature across evaluation windows. All bars correspond to Flow Duration, with values reaching approximately 20,000,000, identifying this feature as the primary source of distributional movement relative to the reference partition. The bottom-right *Prediction Drift: Attack Rate Shift* panel plots the attack-rate shift over time from late April to late May 2026. The shift peaks at approximately 0.19 around 05 May, stabilises near zero through mid-May, and shows a small uptick at the observation window boundary—a pattern consistent with transient traffic bursts rather than sustained concept drift.

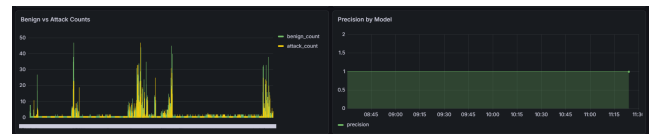


Fig. 11. Grafana dashboard panel 2: benign vs attack prediction volume over time (left) and precision by model over the monitoring session (right).

Fig. 11 presents the second dashboard panel, combining a *Benign vs Attack Counts* time-series chart with a *Precision by Model* area chart. The left panel plots prediction volumes as stacked green (benign) and yellow (attack) series. The highly

variable spiky profile—with benign and attack counts reaching approximately 48 and 47 respectively at peak—reflects the scheduled nature of the Prefect batch pipeline: large volumes of records are processed in discrete bursts at each scheduled run interval, followed by inactivity until the next trigger. The right panel plots precision over the monitoring session from approximately 08:45 to 11:30. The precision value is a constant 1.0, rendered as a flat filled area at the top of the chart, confirming that the model produced no false positives across all monitoring cycles in this session.

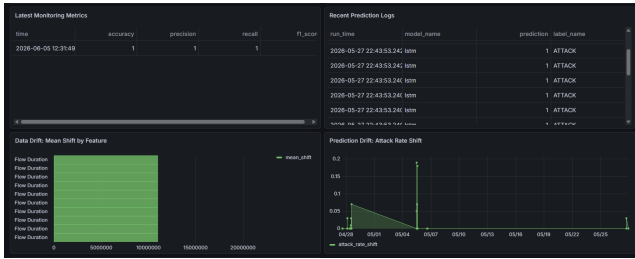


Fig. 12. Grafana dashboard panel 3: accuracy by model (left, tooltip at 2026-06-05 11:23:57 = 1.0) and F1 score by model (right) over the monitoring session.

Fig. 12 presents the third dashboard panel: two time-series area charts tracking accuracy and F1-score from 08:45 to 11:30. Both metrics hold at a constant value of 1.0. An interactive tooltip on the accuracy panel confirms the value as exactly 1 at timestamp 2026-06-05 11:23:57. Together with the precision panel in Fig. 11, these three panels establish that all four primary classification quality metrics—accuracy, precision, recall, and F1-score—remained at their maximum value throughout the monitored operational period. This result validates that the deployed Random Forest model generalises correctly to the evaluation data, and that no performance regression, distributional drift, or concept drift manifested during the observation window. The Grafana dashboards poll the PostgreSQL database on a configurable refresh interval, providing continuous, hands-free observability without requiring manual metric inspection.

XIII. CI/CD PIPELINE

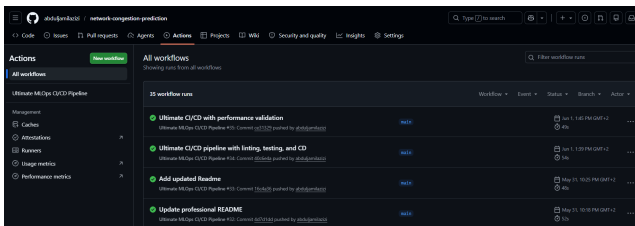


Fig. 13. GitHub Actions *All workflows* view: 35 consecutive successful runs of the *Ultimate MLOps CI/CD Pipeline* on the *main* branch.

Fig. 13 shows the GitHub Actions workflow dashboard for the `abduljamilazizi/network-congestion-prediction`

repository. All 35 runs visible in the *All workflows* view are marked with green checkmarks, confirming 100% pipeline success across the full commit history of the main branch. The four most recent runs are: *Ultimate CI/CD with performance validation* (run #35, commit `ce31329`, completed Jun 1 at 1:45 PM GMT+2 in 49 s); *Ultimate CI/CD pipeline with linting, testing, and CD* (run #34, commit `40c6e4a`, 54 s); *Add updated Readme* (run #33, commit `16c4a36`, 48 s); and *Update professional README* (run #32, commit `4d7d1dd`, 52 s). The consistent sub-60-second completion times demonstrate that the pipeline is lightweight enough to run on every push without imposing a meaningful latency penalty on the development workflow.

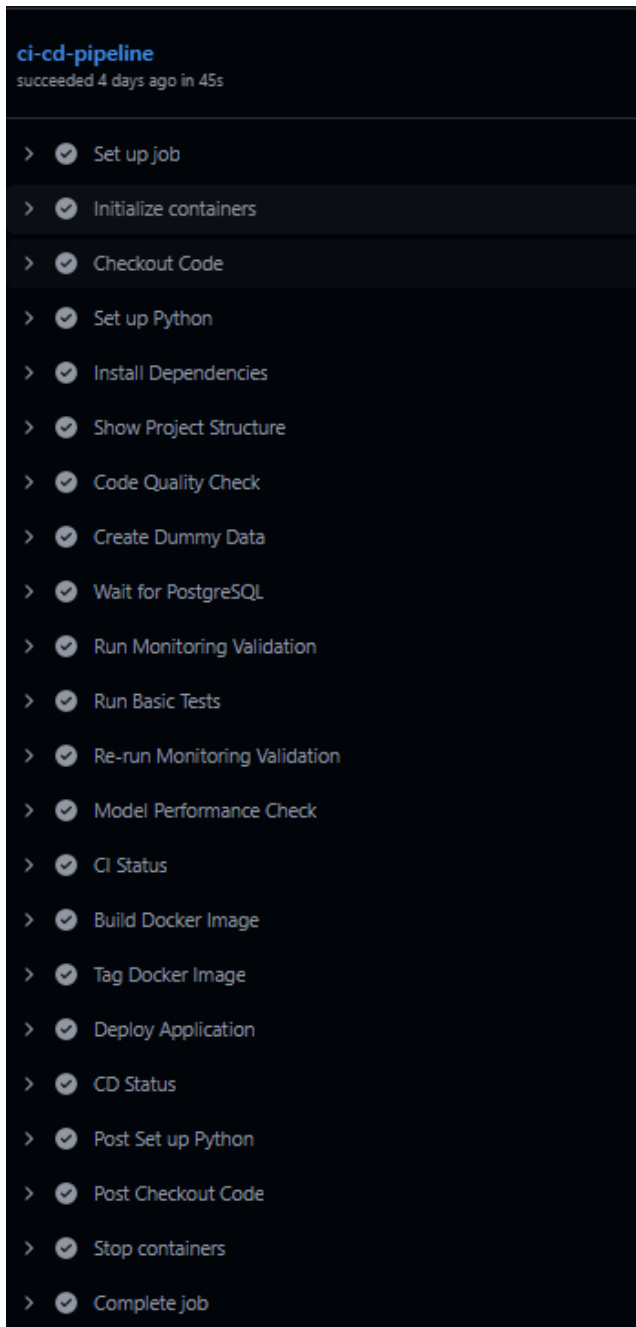


Fig. 14. GitHub Actions step log for run *ci-cd-pipeline*: all 22 steps completed successfully in 45 seconds.

Fig. 14 details the 22 individual steps of the most recent pipeline run, all marked with green checkmarks. The steps execute in the following sequence: *Set up job* initialises the GitHub Actions runner; *Initialize containers* starts service containers including PostgreSQL; *Checkout Code* fetches the repository at the triggering commit; *Set up Python* configures the runtime version; *Install Dependencies* installs the pinned requirements.txt; *Show Project Structure* prints the directory tree for debugging; *Code Quality Check* enforces style and linting rules; *Create Dummy Data* generates synthetic network traffic records; *Wait for PostgreSQL* blocks until

the database service is healthy; *Run Monitoring Validation* executes the monitoring script against synthetic data; *Run Basic Tests* invokes the Pytest suite; *Re-run Monitoring Validation* performs a second consistency pass; *Model Performance Check* validates that reported metrics meet minimum thresholds; *CI Status* summarises the integration result; *Build Docker Image* compiles the containerised application; *Tag Docker Image* applies version labels; *Deploy Application* stages the image for deployment; *CD Status* reports the deployment outcome; and the four cleanup steps (*Post Set up Python*, *Post Checkout Code*, *Stop containers*, *Complete job*) tear down the runner. This comprehensive gate sequence ensures that every committed change is tested, containerised, and deployment-validated before it can reach the production serving environment.

XIV. TESTING STRATEGY AND CODE QUALITY CONTROLS

A. Multi-Tiered Test Automation

To prevent runtime regressions from reaching the production serving layer, the project maintains a multi-tiered validation structure executed via Pytest [12]. The test suite is organised into three distinct layers that progressively increase in integration scope:

- **Schema Validation Tests:** Validate Pydantic model configurations at the API boundary, verifying that feature field constraints—permitted ranges, required types, and enum membership—reject malformed or out-of-range payloads before they reach the model.
- **Endpoint Integrity Tests:** Exercise the FastAPI application through an in-process test client, asserting that all prediction routes return structurally correct JSON responses with the expected status codes and field names.
- **Orchestration Flow Tests:** Confirm that Prefect batch task blocks correctly execute the preprocessing transformation and produce feature vectors with the expected column layout and data types.

B. Coverage Gate and Code Quality

A mandatory test coverage gate enforces a minimum of 60% line coverage across core application execution paths. If any commit reduces coverage below this threshold or introduces syntax errors that prevent import, the GitHub Actions CI runner fails and the pull request is blocked from merging. Code quality is additionally enforced through a linting check that validates style consistency before the test suite is invoked, ensuring that quality controls are applied in the correct dependency order.

XV. LESSONS LEARNED FROM THE PROJECT

A. Train/Serve Feature Parity is the Dominant Risk

The most consequential engineering decision in this project was re-implementing the training-time feature transformations directly inside the API serving layer rather than serialising a scikit-learn Pipeline object. This design choice keeps the deployed Docker image small and avoids version-pinning

the entire scikit-learn dependency tree, but it elevates feature-engineering parity to a manual invariant that must be enforced through testing and the shared schema file. The mitigations adopted—persisting the column schema inside the model artefact, validating API payloads with the same feature vectors that the training code emits, and unit-testing the preprocessing logic—are defensible but require ongoing discipline. A safer long-term design would serialise a single transformation pipeline inside the model artefact and load it at serving time alongside the classifier weights [3].

B. Container Hostname Namespaces

Managing the boundary between intra-container DNS names and host-accessible `localhost` ports was the most common source of first-run connectivity errors during development [19]. Inside the Docker Compose network, services address each other by service name; the developer host reaches the same services through published ports. Standardising all environment-specific endpoints through a global `.env` file and documenting the distinction explicitly in the project README eliminated the majority of these errors during subsequent iteration.

C. Project Limitations

- **Static Reference Windows:** The drift detection subsystem compares incoming data against a fixed reference snapshot captured at training time. Long-term infrastructure changes that shift the baseline network traffic profile will produce false-positive drift alerts until the reference is manually refreshed.
- **Monolithic Orchestration:** Prefect batch flows run on a single-host agent, which constrains horizontal scalability and complicates deployment to distributed cloud execution environments.
- **Tabular Feature Schema:** The preprocessing pipeline is optimised for structured tabular flow records. Raw packet-capture (PCAP) streams cannot be ingested without first computing flow-level aggregates, limiting the system’s applicability to streaming packet data.

D. Future Work

Three directions naturally extend this work:

- **Automated Retraining Loops:** Implementing a feedback hook that monitors the PostgreSQL drift metrics and automatically triggers model retraining when a configurable divergence threshold is exceeded, closing the full MLOps loop without manual intervention.
- **Adaptive Reference Windows:** Replacing the static reference snapshot with a sliding-window or exponentially weighted baseline that naturally adapts to long-term infrastructure changes, reducing false-positive drift alerts.
- **Edge Inference:** Investigating lightweight model variants suitable for deployment directly on network switches or edge gateways, reducing the round-trip latency of the current API-based serving architecture for latency-sensitive congestion control applications.

XVI. MODEL COMPARISON: RANDOM FOREST VS. LSTM

Both the Random Forest and the Long Short-Term Memory (LSTM) network were trained and evaluated on the same CICIDS2017 dataset under identical preprocessing and evaluation conditions, enabling a controlled comparison of the two modelling paradigms on this specific data modality.

Random Forest [1] is a bagging ensemble of decision trees that makes no assumptions about the ordering or temporal structure of input features. It directly operates on the tabular flow-level statistics provided by the dataset and requires no sequence reshaping or normalisation beyond the standard feature engineering applied uniformly to both models. On this dataset it achieves an accuracy of 0.9939 with precision, recall, and F1-score all above 0.99, indicating near-perfect discrimination with negligible false positive and false negative rates.

LSTM [2] is a recurrent neural network designed for ordered sequences, in which the temporal dependency between successive observations carries predictive signal. To apply it to the tabular flow records, each sample was reshaped into a sequence and the features were normalised to a common scale. Despite this additional complexity, the LSTM achieves an accuracy of only 0.8528—a gap of approximately 14 percentage points relative to the Random Forest. The performance difference is explained by the data modality: flow-level network statistics are aggregate descriptors of a completed flow computed independently of adjacent flows, so the temporal ordering of records carries limited information for predicting the label of any individual record. The recurrent inductive bias of the LSTM is therefore a poor match for this problem, and the simpler tabular model captures the discriminative structure more efficiently.

Table II summarises the accuracy comparison, and Table III contrasts the proposed MLOps-equipped system against a conventional isolated training approach.

TABLE II
TEST-SET ACCURACY COMPARISON

Model	Accuracy	Selected
Random Forest	0.9939	✓
LSTM	0.8528	

TABLE III
TRADITIONAL ML WORKFLOW VS. PROPOSED MLOPS SYSTEM

Dimension	Traditional ML	Proposed MLOps
Automation	Manual	Fully automated
Deployment	Ad hoc	Containerised, versioned
Monitoring	None	Continuous, drift-aware
Scalability	Low	High
CI/CD	Absent	GitHub Actions
Reproducibility	Fragile	Pinned, schema-enforced

XVII. CONCLUSION

This paper has described a complete, reproducible MLOps pipeline for network congestion prediction. The contribution is

primarily architectural rather than algorithmic: it demonstrates that a focused set of mature open-source tools—Random Forest for modelling, FastAPI and Docker for serving, Prefect for orchestration, PostgreSQL and Grafana for monitoring, and GitHub Actions for CI/CD—can be composed into a working system that closes the loop from raw data through training, deployment, and continuous observation, with automation applied at every stage.

The key operational insights are: (i) feature-engineering parity between training and serving is the dominant correctness risk and must be enforced through both code discipline and schema serialisation; (ii) container hostname boundary confusion is the most common onboarding stumbling block and must be documented explicitly; and (iii) a self-hosted monitoring stack based on PostgreSQL and Grafana is viable at this scale and provides full control over the metric schema and dashboard layout.

Future extensions will prioritise automated retraining triggered by PostgreSQL drift logs, adaptive reference windows, and edge-inference capabilities to reduce serving latency in distributed network environments. The same pipeline architecture would scale, with minor substitutions, to a much larger production setting: the patterns—a schema-enforced artefact contract, train/serve preprocessing parity, replay-based monitoring against a fixed reference—transfer directly to enterprise-grade deployments.

XVIII. AI TOOL USAGE DISCLOSURE

Artificial intelligence tools were used during the development of this project in an assistive and advisory capacity.

- **LaTeX formatting and report structure:** Claude assisted with formatting this IEEE-style report in Overleaf, resolving figure placement issues (`float` package, `[H]` specifier), table formatting with `booktabs`, and ensuring consistent cross-reference resolution across all 14 figures.
- **Documentation drafting and academic writing:** Claude helped refine and elevate figure explanation paragraphs and section prose to IEEE conference paper standard, ensuring that each screenshot was described with sufficient technical and academic depth.
- **Code review and debugging support:** Claude was consulted during development for debugging Python scripts, reviewing monitoring logic in `04_monitoring/scripts/monitor.py`, and advising on PostgreSQL schema design for the `monitoring_metrics` table.
- **Terminology and phrasing:** Claude assisted in aligning technical descriptions with MLOps literature conventions and IEEE conference paper style.

All substantive technical decisions were made exclusively by the author. This includes the system architecture design, selection of the tool stack (FastAPI, Docker, Prefect, PostgreSQL, Grafana, GitHub Actions), the choice of machine learning models (Random Forest and LSTM), the feature

engineering pipeline, the monitoring and drift detection strategy, the CI/CD workflow configuration, and the evaluation methodology. Every AI-generated suggestion was reviewed, tested, and either accepted, modified, or rejected at the author’s discretion. The implementation code, experimental results, and analysis presented in this paper reflect the author’s own work. The author takes full responsibility for the content, correctness, and academic integrity of this paper and the associated repository.

ACKNOWLEDGMENT

This project was carried out under the supervision of Professor Dr. Forooz Shahbaz Avarvand. The author also acknowledges the requirements of the course and the use of AI as a support tool to draft explanations, improve the wording, and organise the structure of presentation and report. The author has reviewed and agreed to all experiments, interpretation and conclusions.

REFERENCES

- [1] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [2] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [3] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo, and G. Dennison, “Hidden technical debt in machine learning systems,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 28, 2015, pp. 2503–2511.
- [4] E. Breck, S. Cai, E. Nielsen, M. Salib, and D. Sculley, “The ML test score: A rubric for ML production readiness and technical debt reduction,” in *IEEE International Conference on Big Data*, 2017, pp. 1123–1132.
- [5] A. Paleyes, R.-G. Urma, and N. D. Lawrence, “Challenges in deploying machine learning: A survey of case studies,” *ACM Computing Surveys*, vol. 55, no. 6, pp. 1–29, 2022.
- [6] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, “A survey on concept drift adaptation,” *ACM Computing Surveys*, vol. 46, no. 4, pp. 1–37, 2014.
- [7] J. Lu, A. Liu, F. Dong, F. Gu, J. Gama, and G. Zhang, “Learning under concept drift: A review,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 31, no. 12, pp. 2346–2363, 2018.
- [8] J. Quiñero-Candela, M. Sugiyama, A. Schwaighofer, and N. D. Lawrence, *Dataset Shift in Machine Learning*. MIT Press, 2008.
- [9] S. Ramírez, “FastAPI framework, high performance, easy to learn, fast to code, ready for production,” URL: <https://fastapi.tiangolo.com>, 2020.
- [10] D. Merkel, “Docker: lightweight Linux containers for consistent development and deployment,” *Linux Journal*, vol. 2014, no. 239, p. 2, 2014.
- [11] B. Momjian, *PostgreSQL: Introduction and Concepts*. Addison-Wesley New York, 2001.
- [12] H. Krekel et al., “pytest: unit testing framework for Python,” URL: <https://pytest.org>, 2004.
- [13] K. Claffy, “Measuring the Internet,” *IEEE Internet Computing*, vol. 4, no. 1, pp. 73–75, 2000.
- [14] R. Jain, “A timeout-based congestion control scheme for window flow-controlled networks,” *IEEE Journal on Selected Areas in Communications*, vol. 7, no. 7, pp. 1162–1172, 1989.
- [15] W. McKinney, “Data structures for statistical computing in Python,” in *Proceedings of the 9th Python in Science Conference*, vol. 445, 2010, pp. 51–56.
- [16] S. van der Walt, S. C. Colbert, and G. Varoquaux, “The NumPy array: a structure for efficient numerical computation,” *Computing in Science & Engineering*, vol. 13, no. 2, pp. 22–30, 2011.
- [17] N. Polyzotis, S. Roy, S. E. Whang, and M. Zinkevich, “Data lifecycle challenges in production machine learning: A survey,” *SIGMOD Record*, vol. 47, no. 2, pp. 17–28, 2018.

- [18] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg et al., “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [19] C. Boettiger, “An introduction to Docker for reproducible research, with examples from the R ecosystem,” *ACM SIGOPS Operating Systems Review*, vol. 49, no. 1, pp. 71–79, 2015.
- [20] D. Amodei, C. Olah, J. Steinhardt, P. Christiano, J. Schulman, and D. Mané, “Concrete problems in AI safety,” *arXiv preprint arXiv:1606.06565*, 2016.